```
        a[i] = i;
        b[i] = 2 * i;
    }

    // allocate the memory on the GPU
    cudaMalloc( (void**)&dev_a, N * sizeof(int) ) ;
  cudaMalloc( (void**)&dev_b, N * sizeof(int) );
cudaMalloc( (void**)&dev_c, N * sizeof(int) );

     // copy the arrays 'a' and 'b' to the GPU
    cudaMemcpy( dev_a, a, N * sizeof(int),cudaMemcpyHostToDevi
ce );
    cudaMemcpy( dev_b, b, N * sizeof(int),cudaMemcpyHostToDevice
);

    vector_add<<<numBlock,numThread>>>( dev_a, dev_b, dev_c );

    // copy the array 'c' back from the GPU to the CPU
    cudaMemcpy( c, dev_c, N * sizeof(int),cudaMemcpyDeviceToHost
) ;

//Prints the results
for (int i=0; i<N; i++)
{
printf("%d %d %d \n\n", a[i],b[i],c[i] );
}

     // free the memory we allocated on the CPU
    free( a );
    free( b );
    free( c );

   // free the memory we allocated on the GPU
   cudaFree( dev_a ) ;
   cudaFree( dev_b ) ;
    cudaFree( dev_c);

    return 0;
}
```

Let's begin with analysing each part of the code, and then compile the code to get our results.

From the previous article (Part 1 of this series), you know that CUDA programs execute in two places—the host (your CPU) and the device (GPU). You might be a bit surprised by the fact that writing the device code is much simpler than writing the CPU host code. Hence, let's begin with analysing the device code first.

Since we have 100 array values in this code, to simplify things, let's have 100 blocks (kernels) running simultaneously, where each kernel runs a single thread. Hence, let's set numThread to 1 and numBlock to 100, and use these variables later while calling the device from the host.

The device code is:

```
__global__ void vector_add(int *a, int *b, int *c)
     // keep track of the index
    int tid = blockIdx.x;

    while (tid < N) {
        c[tid] = a[tid] + b[tid];
        tid = tid+ numBlock; // shift by the total number of
blocks, i.e. 100 in our case
    }
}
```

As shown above, add a *__global__* qualifier to the function *vector_add*. Notice that there are very few changes in the function *vector_add* of the serial and parallel sections. The *__global__* qualifier indicates that this is a device function that would be called from the host.

blockIdx is a built-in CUDA runtime variable, which is a three-component vector to identify threads in a one, two and three dimension index. Imagine a block as a 3-D matrix and to access the different components in this vector, use *blockIdx.x*, *blockIdx.y* and *blockIdx.z*. In this code, we will be using 100 blocks with a single thread on every grid, which will be seen while we analyse the host code, and hence we use only *blockIdx.x* which returns the current block number.

The condition *while tid<N* checks that the bounds for array computation have not been reached and computes the array sum taking the block value as an index, i.e., *tid*.

Add *numBlock* to the *tid* value, to shift the index by the number of blocks, as each block would be computing just one array index, and we have 100 blocks for 100 array indexes. This explanation pretty much sums up the device code for the program.

Now move on to the host code, which prepares the GPU for execution and invokes the kernel. It works by allocating memory to the GPU and CPU, transfers the input vectors to the GPU, launches the kernel and transfers the result back to the host (CPU).

```
int main( void ) {
    int *a, *b, *c;
    int *dev_a, *dev_b, *dev_c;

    // allocate the memory on the CPU
    a = (int*)malloc( N * sizeof(int) );
    b = (int*)malloc( N * sizeof(int) );
    c = (int*)malloc( N * sizeof(int) );

    // fill the arrays 'a' and 'b' on the CPU
   for (int i=0; i<N; i++) {
        a[i] = i;
       b[i] = 2 * i;
    }

    // allocate the memory on the GPU
   cudaMalloc( (void**)&dev_a, N * sizeof(int) ) ;
   cudaMalloc( (void**)&dev_b, N * sizeof(int) );
```